

Secure Instant Point-to-Point Messaging

CS 5173 — Computer Security
Spring 2026

Colby Frison
Johnpaul Nguyen
Jacob Woolbright

May 9, 2026

Contents

1	Introduction	2
2	System Overview	2
2.1	Architecture	2
2.2	Concurrency model	3
3	Cryptographic Design	3
3.1	Cipher and key length	3
3.2	Authenticated key agreement	3
3.3	Per-message nonce and ciphertext uniqueness	4
3.4	Padding	5
4	Key Management and Rotation	5
5	Networking and Connection Management	5
6	GUI Design	6
6.1	Interface layout and message bubbles	6
6.2	Dialogs and settings	6
6.3	Command-line interface	6
7	Security Analysis	7
7.1	What the system protects	7
7.2	Known limitations	7
7.3	Potential future improvements	7
8	Testing and Validation	8
8.1	Automated integration testing	8
8.2	Manual GUI verification	8
9	Conclusion	8
10	Program Screenshots	9

Abstract

This report presents the design and implementation of a secure point-to-point messaging system. The project satisfies all requirements stated in the assignment, which include: encrypted messaging, a desktop GUI displaying ciphertext and plaintext for each message, fresh ciphertext on repeated sends, and rotating keys. As stated in the bonus section, instead of relying on a pre-shared passphrase, the system performs authenticated key agreement using ephemeral ECDH and Ed25519 signatures, then derives a 256-bit AES-GCM session key with HKDF. This report explains each design decision, justification of each decision, and how each component interacts to produce a coherent and verifiable security model.

1 Introduction

Instant messaging tools are typically a very common target for attacks, as their simplicity makes eavesdropping, message replay, and tampering very easy. A secure chat application, therefore, must guarantee the confidentiality of message content, detect any tampering to transmitted messages, and prevent an attacker from learning anything useful from the ciphertext. At the same time, the system must remain easily usable for ordinary users, which is why the GUI is present: making the interactions simple, and displaying the cryptography instead of making it opaque.

In this project, we implemented a two-party secure chat application using Python. The networking layer uses the standard socket library for TCP communication, and all cryptographic primitives are provided by the cryptography package. The desktop interface was first modeled in Figma, then made with Tkinter, using CustomTkinter for some elements. Together, these components comply with all core requirements listed in the assignment: a key of length at least 56, proper key derivation, per-message nonce randomization so plaintexts yield distinct ciphertexts, GUI display of both sent and received ciphertext and plaintext, and rotating keys.

2 System Overview

2.1 Architecture

The codebase has three main layers, the first of which is the transport and cryptography layer (`simple_socket.py`), which does socket connections, performs the authentication handshake, encrypts and decrypts messages, manages session keys, and implements the key rotation. The controller layer (`chat_controller.py`) acts as an intermediary layer between the front and backend: it starts and restarts listeners, dispatches calls to threads, manages the message queue, and handles state changes back onto the Tk main thread with `root.after`. The GUI layer (`gui/chat_gui.py`) contains all user-facing logic: connect and Settings dialogs, message bubbles, and the connection status.

This separation was made to make the development process easier to divide amongst the group, but it also makes development easier if we decide to continue the project and create a larger-scale system.

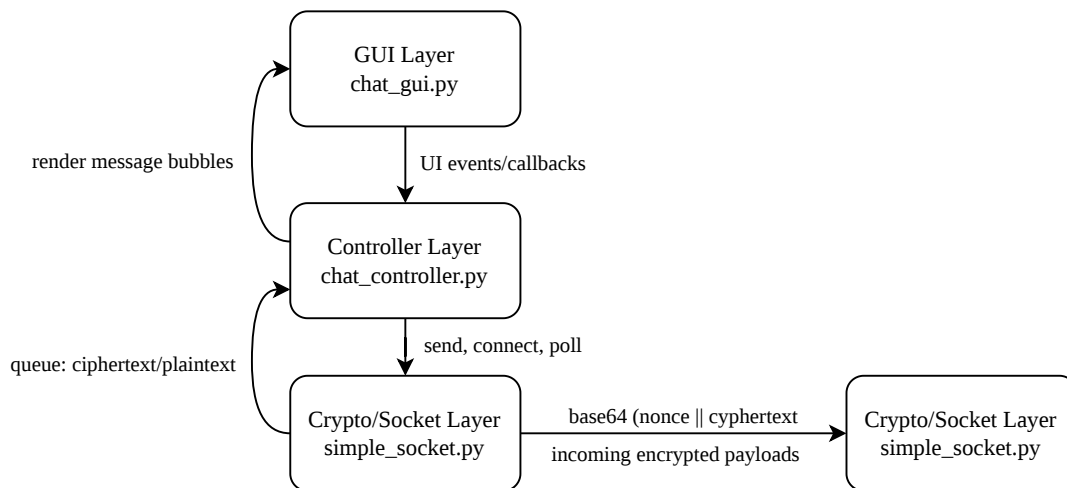


Figure 1: Three-layer architecture and data flow.

2.2 Concurrency model

Every network call listens for connections, performs the handshake, and executes the per-connection receive loop, which runs in a background daemon thread. The controller layer uses two primary mechanisms to return results safely to the GUI thread. Event hooks (`on_peer_connected`, `on_key_rotated`, etc.) post status strings with `root.after(0, ...)`, which schedules execution on the next Tk event loop iteration. Incoming decrypted messages are placed on a Python queue by the receive thread and drained every 150 ms by a `root.after` poll loop running on the main thread. Neither path ever blocks the GUI.

3 Cryptographic Design

3.1 Cipher and key length

The system uses AES-GCM (Galois/Counter Mode) with a 256-bit key. AES-GCM was chosen because it well exceeds the 56-bit key requirement, and it is an Authenticated Encryption with Associated Data (AEAD) scheme, meaning a single encrypt call produces the ciphertext and a message authentication code. Any tampering with the ciphertext during transmission causes decryption to raise an error and the message to be discarded, making the system have both confidentiality and integrity in a single primitive without requiring a separate HMAC step. AES-GCM also has hardware acceleration on pretty much every modern processor, keeping latency very low for our application.

3.2 Authenticated key agreement

Instead of having users agree on a shared password, the system negotiates a session key while authenticating both sides. The following is a layout of how the protocol works:

- On startup, each peerNode generates a long-term *Ed25519* identity key pair that persists for the

lifetime of the process.

- When a connection is established, each side independently generates a fresh ECDH key pair on curve P-384 and signs the ephemeral public key with its Ed25519 identity key.
- The signed payload (identity public key, ephemeral public key, and signature) is then transmitted over the TCP socket as a JSON object with base64-encoded fields.
- The receiver verifies the signature before proceeding, a failed verification immediately aborts the connection.
- Both sides then perform an ECDH exchange over their respective ephemeral keys and pass the resulting shared secret through HKDF-SHA-256 with a fixed info string to produce the 256-bit AES session key:

$$K_{AB} = \text{HKDF-SHA256}(\text{ECDH}(e_A, e_B), \text{"p2p-secure-handshake"})$$

Using ephemeral keys for the exchange means that even if an identity key is later compromised, past session keys cannot be reconstructed, providing a degree of forward secrecy at the session level.

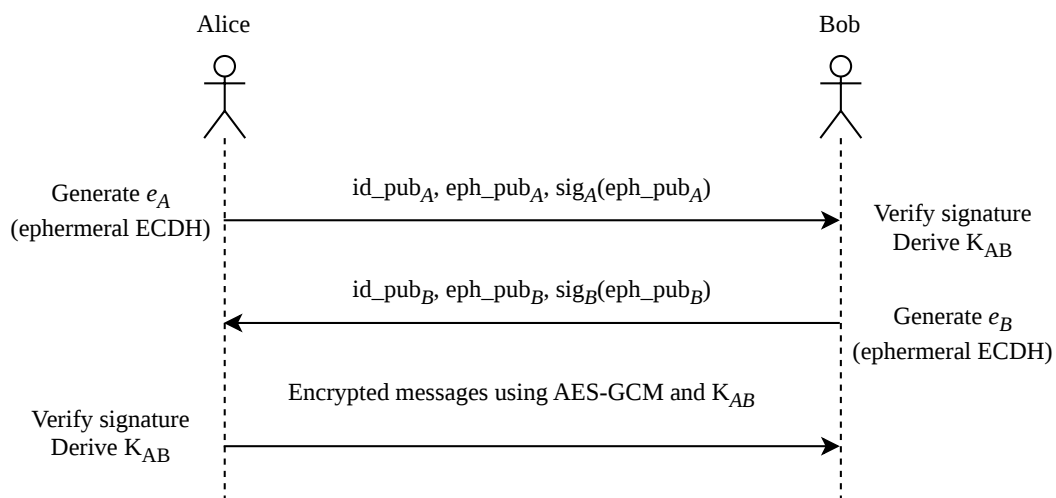


Figure 2: Authenticated handshake and session key derivation.

3.3 Per-message nonce and ciphertext uniqueness

A common requirement for symmetric ciphers is ensuring identical plaintexts does not produce identical ciphertexts, this makes sure an eavesdropper cannot detect when the same word or phrase is repeated. This is satisfied by generating a 12-byte nonce from the operating system's random source function (`os.urandom(12)`) for every individual message. The transmitted payload takes the form:

$$\text{payload} = \text{base64}(\text{nonce} \parallel \text{ciphertext})$$

The receiver splits off the leading 12 bytes (the nonce) and passes the remainder to AES-GCM decrypt. Because the nonce changes for every message, two encryptions of a given word is guaranteed to produce a different string.

3.4 Padding

AES-GCM is a counter-based stream cipher mode and does not process data in fixed-size blocks, so no manual padding is required or applied during encryption or decryption. This is an advantage over CBC mode, which requires careful padding and is susceptible to attacks when implemented incorrectly.

4 Key Management and Rotation

The session key derived at handshake time is not used throughout the entire conversation. After every five messages sent or received on a given connection, both peers independently apply a one-way HKDF ratchet to derive a new successor key:

$$K_{i+1} = \text{HKDF-SHA256}(K_i, \text{"key-rotation-ratchet"})$$

The previous key is discarded, and the message counter is reset to zero. Both sides perform this derivation identically so they remain synchronized without any additional communication.



Figure 3: One-way key evolution via HKDF ratchet.

This design provides three concrete security benefits. First, it bounds the data volume encrypted under any single key, reducing the amount of ciphertext an attacker can collect to crack one key. Second, it provides partial forward secrecy: if a session key K_i is extracted from memory, it can be used to decrypt at most the five messages sent under it, and HKDF's one-way nature means K_i cannot be reversed to obtain K_{i-1} or earlier messages. Third, it ensures **forward key progression**: even with K_i in hand, the attacker cannot predict K_{i+1} without executing the HKDF themselves on messages they do not possess. The GUI also shows rotations just displaying a small status message displaying “Key rotated” and the peer address whenever a ratchet step occurs.

5 Networking and Connection Management

Each PeerNode instance can act simultaneously as a server (accepting incoming peers) and as a client (dialling a known address). The listener binds a TCP socket and blocks inside a daemon thread. Each accepted connection is handed to a dedicated `handle_client` thread that performs the server side of the handshake and then enters the message receive loop. Outbound connections follow the symmetric

client-side path: a new socket is created, then the handshake is completed, and a receive loop thread is started for that connection too. The session state (the socket, the AES session key, and the message counter) is stored per peer address in dictionaries protected by a threading lock.

The controller layer exposes four optional hook points that are assigned by `ChatController` during setup. When a secure session is established, `on_peer_connected` fires with the peer address and a flag indicating incoming or outgoing. On disconnect, `on_peer_disconnected` is called. After each key rotation, `on_key_rotated` fires with the peer address. The fourth hook, `on_message_received`, is set to a no-op because the inbound queue polling loop drives the GUI bubbles directly, which would keep both paths active in-turn producing duplicate messages. All hook calls are controlled on the Tk main thread by the controller before they update any widget.

6 GUI Design

6.1 Interface layout and message bubbles

The interface presents a sidebar showing the connection state and a main chat area. Each message, whether sent or received, is rendered with the Icon of the user who sent the message with two bubbles containing the ciphertext and plaintext adhering to the assignments requirements.

The outgoing bubble is created only after the `send_message` call returns a non-null ciphertext value, ensuring the displayed ciphertext is the one actually rendered. If encryption fails for any reason (for example, no session key yet established), the status bar shows an error, and no message is added.

6.2 Dialogs and settings

The Connect dialog accepts the peer's host and port and triggers the handshake in a background thread. The Settings dialog allows the user to change their display name, the listen bind address, and the listen port, saving immediately restarts the TCP listener on the new address. Both of these dialogs/menus are modals so the main interface does not have to be overwritten. On Linux systems running (Wayland in our experience), `grab_set` requires the window to be mapped before it can acquire the input grab, to make sure this is applied the application detects a Wayland session and calls `wait_visibility` first. On Windows and X11, this extra step is skipped as it is done natively and can break the interface. The Info button opens the help text from `docs/chat_gui_help.txt` in a read-only scrollable panel, offering a short guide for unfamiliar users.

6.3 Command-line interface

The application accepts four optional arguments at startup: `-listen-host` (default 0.0.0.0), `-listen-port` (default 8001), `-peer-host` (default 127.0.0.1), and `-peer-port` (default 8002). If a `gui_settings.json` file exists in the project, the saved settings override the command-line defaults,

making it simple to run persistent instances. This design makes it very easy for two instances to be started on the same machine for local testing:

```
python src/chat_gui.py --listen-port 8001 --peer-port 8002
python src/chat_gui.py --listen-port 8002 --peer-port 8001
```

7 Security Analysis

7.1 What the system protects

The implemented design addresses the three foundational security properties. **Confidentiality** is guaranteed by 256-bit AES-GCM as no party without the session key can recover plaintext from the transmitted ciphertext. **Integrity** is enforced by the GCM authentication tag appended to every message as any byte modified in transit causes decryption to throw an `InvalidTag` exception and the message will subsequently be dropped. **Authenticity** is provided by Ed25519 signatures over the ephemeral keys as a man-in-the-middle who intercepts and replaces handshake messages cannot forge a valid signature for an identity key the attacker does not possess. Finally, the per-message random nonce ensures **ciphertext non-determinism**, and the HKDF ratchet limits exposure under any single key.

7.2 Known limitations

The current trust model has two notable gaps. First, because long-term Ed25519 keys are generated fresh each time the application starts and are never persisted or compared against a known fingerprint, the system is vulnerable to impersonation on the very first connection if the channel is actively monitored before trust is established (a Trust-on-First-Use problem). Second, the application does not obfuscate metadata, meaning a passive observer can determine the IP addresses of both parties, the timing and rough frequency of their messages, and the approximate plaintext length from the size of each ciphertext.

7.3 Potential future improvements

Several enhancements would address these gaps. The first of which being a key fingerprint display in the Connect dialog, analogous to the “safety number” shown by Signal, allowing users to verify peer identity out of band and detect impersonation. Additionally, Applying fixed-size message padding before encryption would also hide plaintext lengths from length-based traffic analysis. Finally, upgrading the symmetric HKDF ratchet to a full Double Ratchet implementing per-message Diffie-Hellman exchanges would provide post-compromise security, meaning even if a session key is extracted from a running process, the system identifies and solves fixes the issues as soon as new messages are exchanged.

8 Testing and Validation

8.1 Automated integration testing

The repository includes `test/test-msg.py`, a self-contained script that validates the cryptographic backend without a graphical interface. It instantiates two `PeerNode` objects on loopback addresses using high-numbered ports (38471–38474) to avoid collisions, it then starts their listener threads, and orchestrates a full conversational scenario. The first test verifies bidirectional delivery: Alice sends a message to Bob, whose queue is checked for the correct plaintext, Bob then replies to Alice, and Alice’s queue is verified in turn. The second test exercises the key rotation: Alice connects to Bob, sends four messages (one below the rotation threshold), and asserts that the session key is unchanged. After the fifth send, the test asserts that the key has been replaced with a new derived value. All tests are run from the project root with `python test/test-msg.py` and pass in the current implementation.

8.2 Manual GUI verification

The ciphertext uniqueness requirement was verified manually by sending the same one-word message several times in succession and observing that each outgoing bubble displayed a different base64 ciphertext string. This confirms that the OS function call `os.urandom(12)` is producing distinct nonces for each call. The key rotation status message were verified by monitoring the status bar during a test exchange, we would ensure that after the fifth message, the bar displayed the expected “Key rotated” notification. And finally both the sender and receiver bubble layouts were confirmed to show ciphertext and plaintext as required.

9 Conclusion

This project delivers a secure P2P instant messaging prototype that satisfies the system requirements provided in the assignment. Authenticated ECDH key agreement with Ed25519 signatures replaces a static shared passphrase and provides per-session forward secrecy at the handshake level. AES-GCM with a 256-bit key encrypts and authenticates every message, and per-message random nonces ensure that repeated plaintext never produces a repeated ciphertext. A periodic HKDF ratchet limits the lifetime of any individual session key and partially bounds the impact of key exposure. The GUI presents ciphertext and plaintext side by side for both sent and received messages, making the cryptographic process transparent and verifiable. The layered architecture separates concerns cleanly and keeps the interface responsive under all network conditions.

10 Program Screenshots

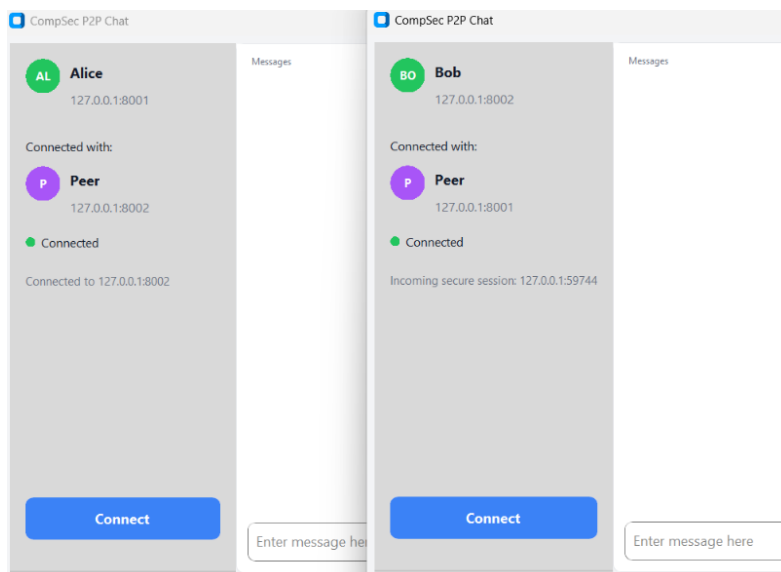


Figure 4: Connection between users

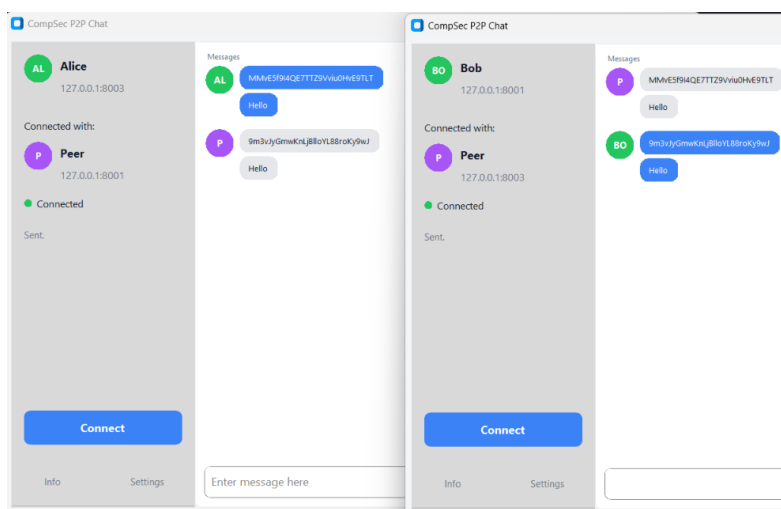


Figure 5: Messaging with Cipher and plain text along with different cipher text for identical plain texts

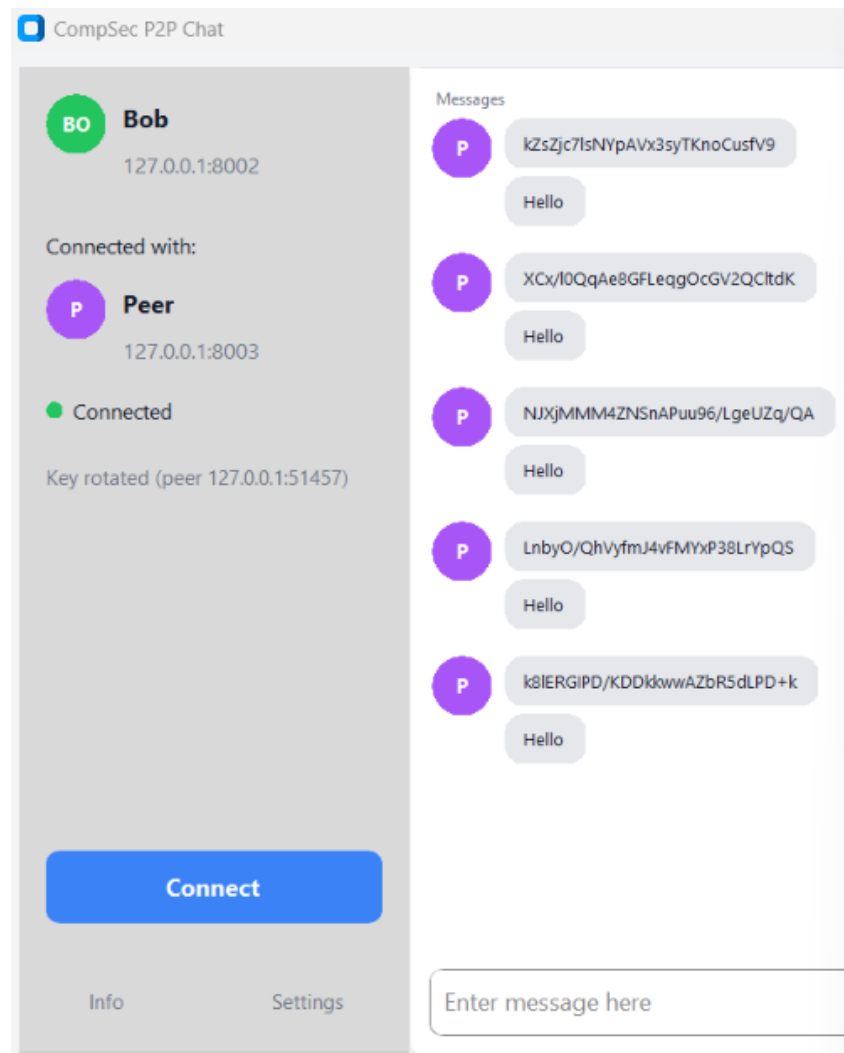


Figure 6: Key rotation after 5 messages